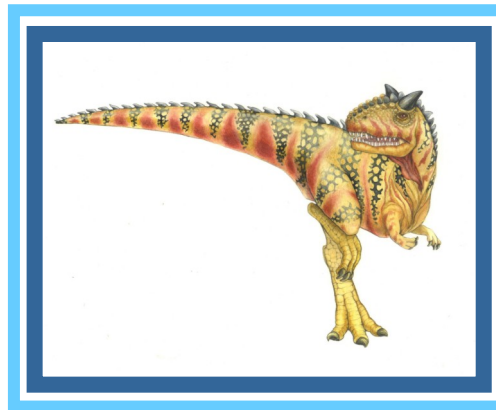


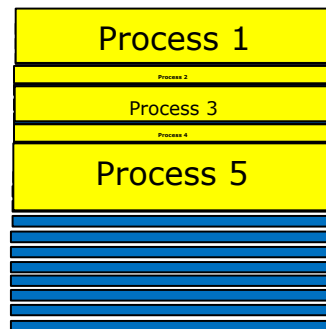
Chapter 8: Memory Management





Fragmentation

- **External Fragmentation** – total memory space exists to satisfy a request, but it is not contiguous
- **Internal Fragmentation** – allocated memory may be slightly larger than requested memory; this size difference is memory internal to a partition, but not being used
- First fit analysis reveals that given N blocks allocated, $0.5 N$ blocks lost to fragmentation
 - $1/3$ may be unusable -> **50-percent rule**



External fragmentation



Internal fragmentation





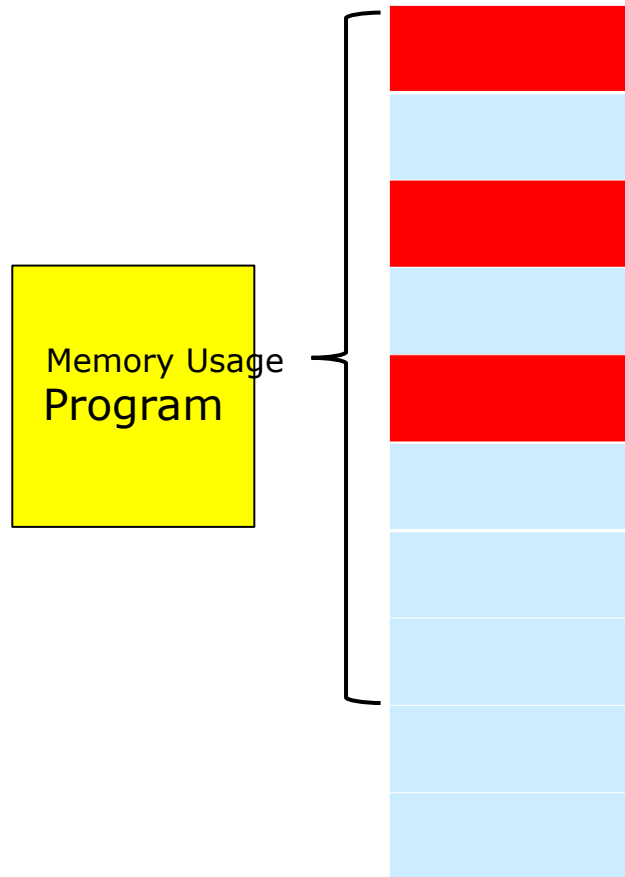
Defragmentation (Cont.)

- Reduce external fragmentation by **compaction or defragmentation**
 - Shuffle memory contents to place all free memory together in one large block
 - Compaction is possible *only* if relocation is dynamic, and is done at execution time
 - Time consuming and all the processes are affected
- Now consider that backing store has same fragmentation problems
- *Any other methods for defragmentation?*
 - Segmentation
 - Paging

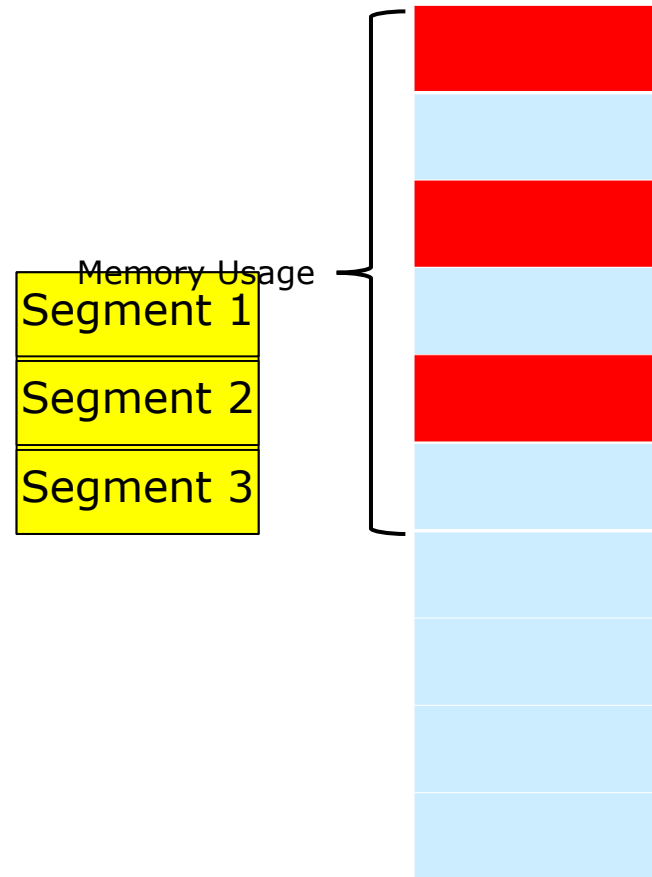




Visualization of Segmentation



No Segmentation



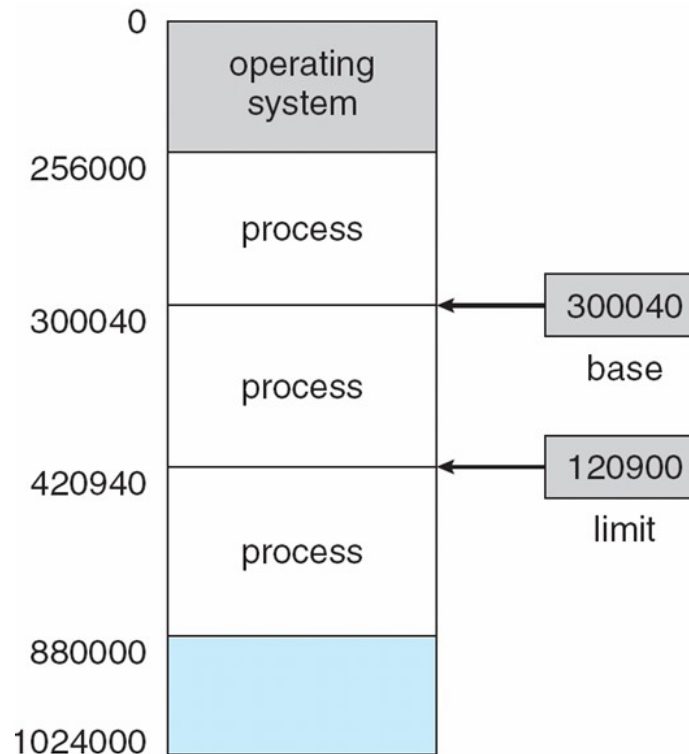
Segmentation





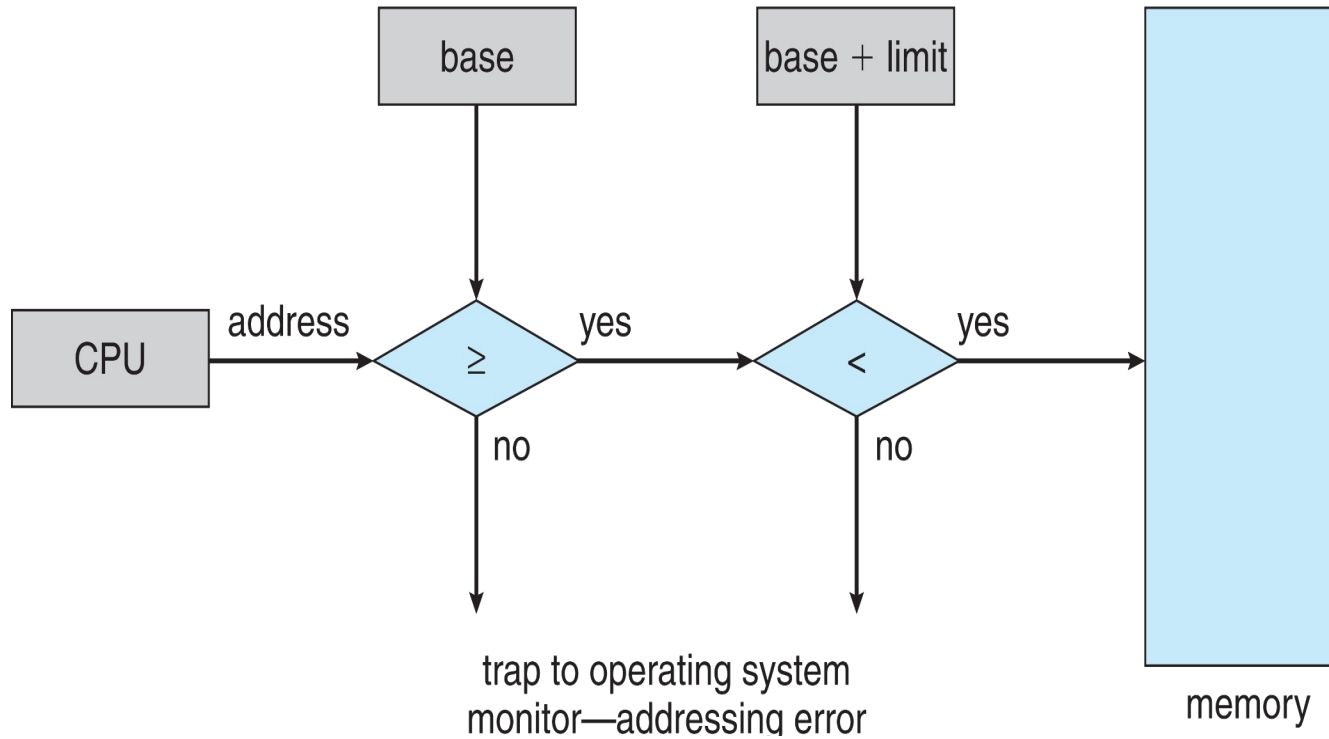
Base and Limit Registers

- A pair of **base** and **limit registers** define the address space
- CPU must check every memory access generated in user mode to be sure it is between base and limit for that user





Hardware Address Protection

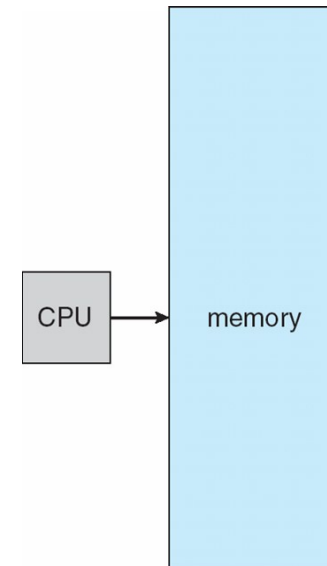




Static Linking

- **Static linking** – system libraries and program code combined by the loader into the binary program image
- *Any limitation?*

- Process may waste the memory resource if they are waiting for I/O
- Memory resource is fixed by the physical memory size
- Defragmentation not allowed





Dynamic Relocation/Linking

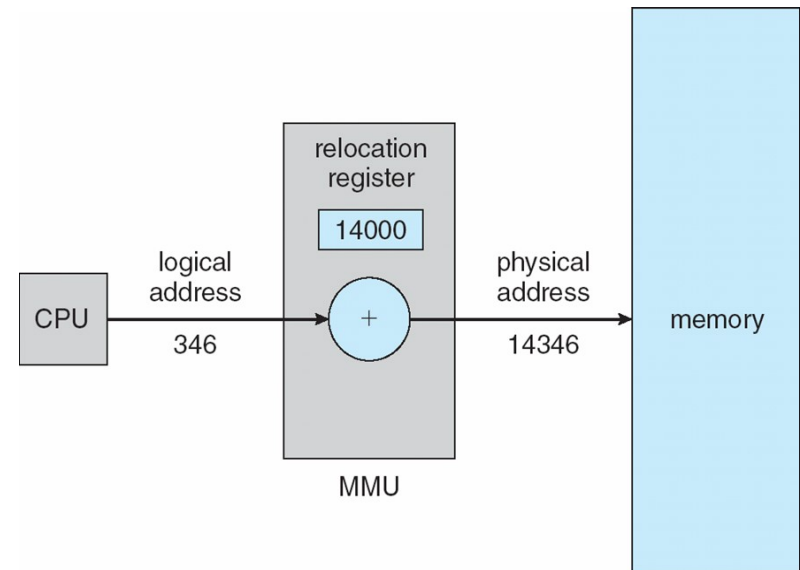
- The concept of a logical address space that is bound to a separate **physical address space** is central to proper memory management
 - **Logical address** – generated by the CPU; also referred to as **virtual address**
 - **Physical address** – address seen by the memory unit
- **Dynamic Relocation/Linking**
 - Dynamic linking postponed until execution time
 - Memory Management Unit: manage the mapping between logical/virtual address and physical address
 - Relocation register is added to every address generated by a user process at the time it is sent to memory
 - **More efficient utilization of memory resource**





Dynamic relocation using a relocation register

- Routine (i.e., the link between process and memory space) is not loaded until it is called
- All routines kept on disk in relocatable load format
- Useful when large amounts of code are needed to handle infrequently occurring cases
- Better memory-space utilization; unused routine is never loaded
 - *How?*





Paging

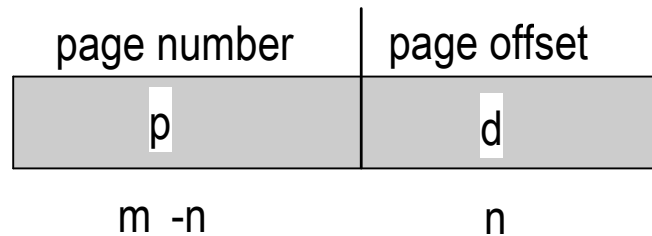
- Physical address space of a process can be noncontiguous; process is allocated physical memory whenever the latter is available
 - Avoids external fragmentation
 - Avoids problem of varying sized memory chunks
- Divide physical memory into fixed-sized blocks called **frames**
 - Size is power of 2, between 512 bytes and 16 Mbytes
- Divide logical memory into blocks of same size called **pages**
- Keep track of all free frames
- To run a program of size ***N*** pages, need to find ***N*** free frames and load program
- Set up a **page table** to translate logical to physical addresses
- Backing store likewise split into pages
- Still have Internal fragmentation





Address Translation Scheme

- Address generated by CPU is divided into:
 - **Page number** (p) – used as an index into a **page table** which contains base address of each page in physical memory
 - **Page offset** (d) – combined with base address to define the physical memory address that is sent to the memory unit

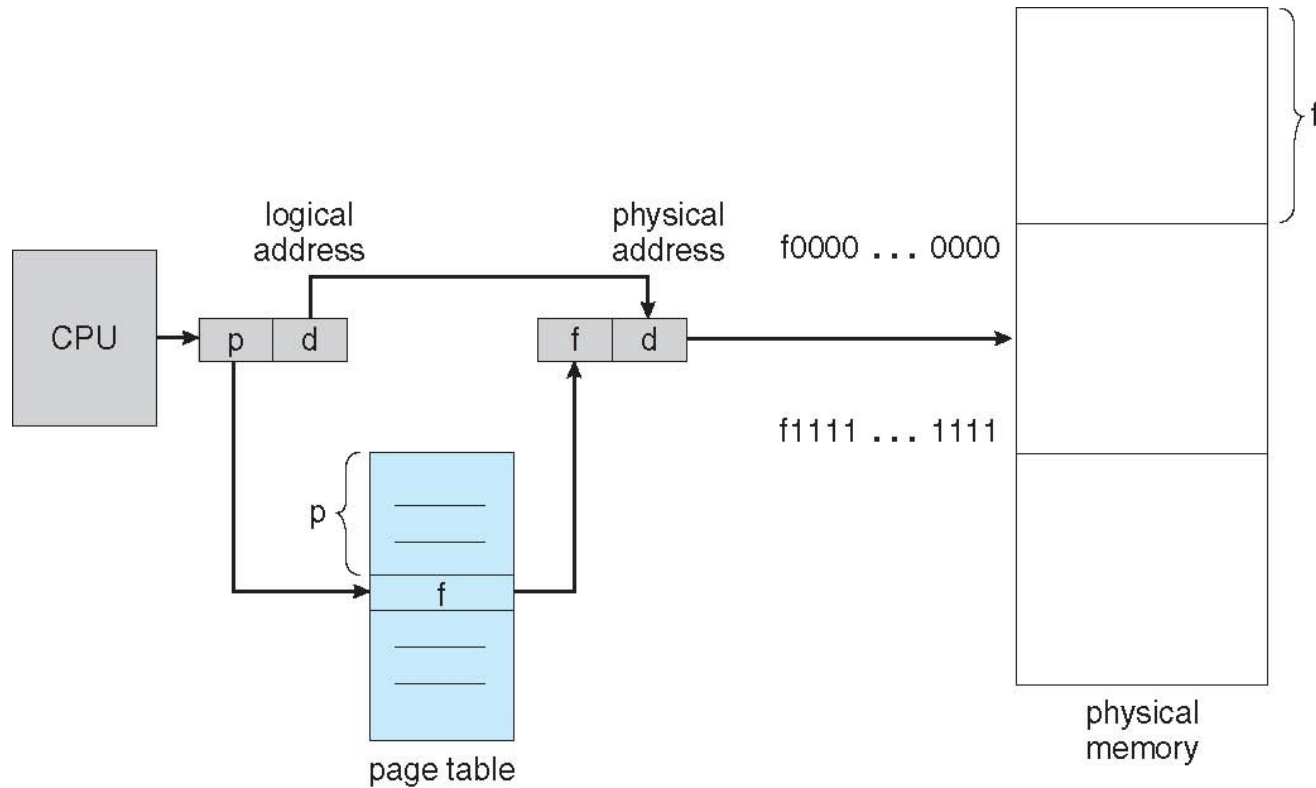


- For given logical address space 2^m and page size 2^n



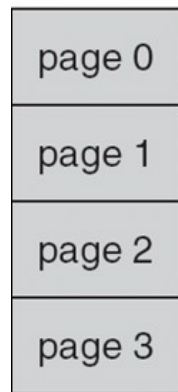


Paging Hardware





Paging Model of Logical and Physical Memory

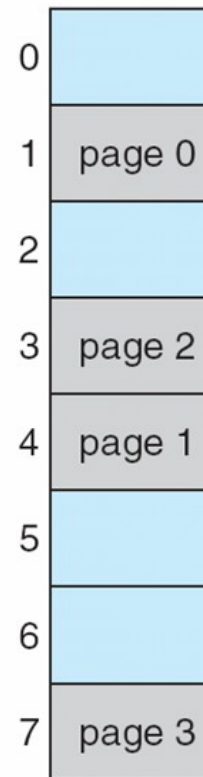


logical
memory

0	1
1	4
2	3
3	7

page table

frame
number



physical
memory





Paging Example

0	a
1	b
2	c
3	d
4	e
5	f
6	g
7	h
8	i
9	j
10	k
11	l
12	m
13	n
14	o
15	p

logical memory

0	5
1	6
2	1
3	2

page table

0	
4	i j k l
8	m n o p
12	
16	
20	a b c d
24	e f g h
28	

physical memory

What is the physical address for variable o?





In Class Practice

- Given the page table as shown below, and the page/frame size of 2^4 . Variable 'a' is assigned with a logical address of 29. What is the physical address of 'a' in the main memory?

0	5
1	6
2	1
3	2

page table

page number p = logical address / page size

offset d = logical address % page size

page number ----page table----> frame number

physical address = frame size * frame number + offset

'a' is at logical address 29, so $p = 1$, $d = 13$

physical frame is 6 according to the page tab

so physical address = $6 * 2^4 + 13 = 109$





Memory Management Cont.

- How to defragment?
 - Segmentation
 - Paging
 - Increase the utilization of physical memory resource
 - Memory allocation more flexible

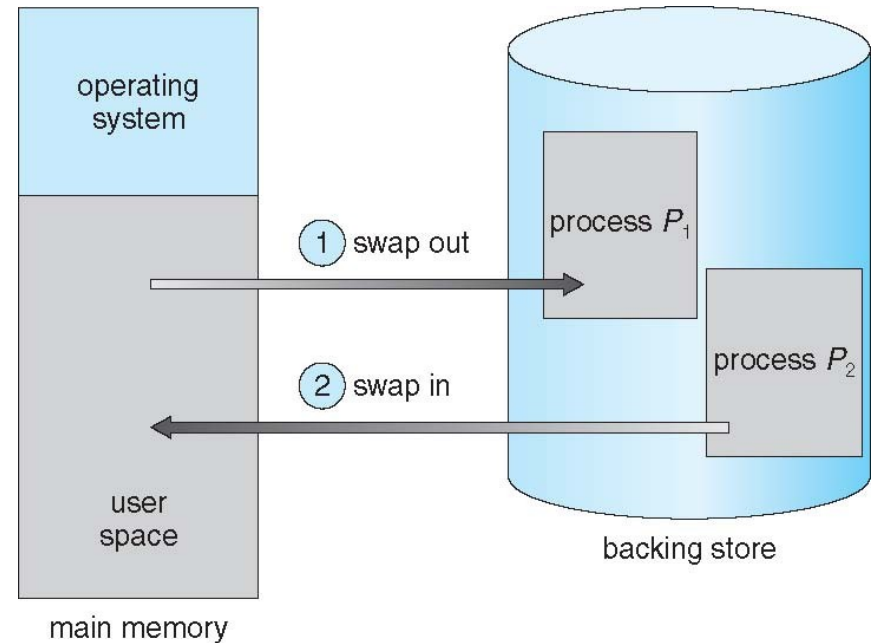
- *But still, the size of physical memory is very limited. How to make it bigger?*
 - Swapping
 - Backing store





Swapping

- A process can be **swapped** temporarily out of memory to a backing store, and then brought back into memory for continued execution
 - *Total physical memory space of processes can exceed physical memory*
- **Backing store** – fast disk large enough to accommodate copies of all memory images for all users; must provide direct access to these memory images
- **Roll out, roll in** – swapping variant used for priority-based scheduling algorithms; lower-priority process is swapped out so higher-priority process can be loaded and executed
- Major part of swap time is transfer time; total transfer time is directly proportional to the amount of memory swapped
- System maintains a **ready queue** of ready-to-run processes which have memory images on disk





Virtual Memory That is Larger Than Physical Memory

